

Last updated: January 24, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 4

Tasks/Questions

1. Compile and setup [3000userlogin](#) as described above. Create a new user account. Verify that you can use 3000userlogin to login as the new user without typing the password of the new user (without logging in beforehand). Think about why you are able to avoid typing the password and still get authenticated.
As 3000userlogin simulates the steps after authentication, “setuid root” is used to bypass authentication as root. Keep in mind that user authentication is just a user-space thing, i.e., after authentication the same setuid will be involved (so we could even say we did not bypass any authentication by replacing it with setuid, but just skipped it and directly went to setuid).
2. Why is the syntax highlighting gone (coloring characters)? What simple change (one line) can you make to have 3000userlogin preserve the colors? (Hint: check the environment variables)
**Your terminal's behavior is determined by what the terminal (emulator) thinks it can do. If you use the command `env` to check, you may notice an environment variable `TERM=xterm-256color`, which is no longer there after 3000login (aside from many others gone). So you can add one line to 3000login: `setenv("TERM", "xterm-256color", 1);`
P.S. if you check `man environ`, you'll see “TERM The terminal type for which output is to be prepared”.**
3. What is returned as the user's password by `getpwnam()` (in terms of type)? Is this what you expected? Read the manual page as you find necessary.
It is the `struct passwd`, corresponding to one line of the file `/etc/passwd`.
4. Compare the uid, gid, euid, egid when running 3000userlogin as a regular user (this means without the “chown” and “chmod” commands above), running it setuid root, and running it as root (i.e., running it from a root shell without the setuid bit set, e.g., “`sudo ./3000userlogin someuser`”). Also check to see what happens when you set the setgid bit.
**regular: uid=1000, euid=1000, gid=1000, egid=1000
setuid root: uid=1000, euid=0, gid=1000, egid=1000
root: uid=0, euid=0, gid=0, egid=0
setgid root: uid=1000, euid=1000, gid=1000, egid=0
setuid only affects euid and running as root will have all ids set to be that of root. Note that the root privilege (the special admin/super user) is only determined by uid/euid but not gid/egid (so gid/egid=0 only says you are in the group called root, not necessarily a superuser).**
5. Why does 3000userlogin change its gid before changing its uid? What happens if you switch the order of these operations?
You won't be able change to another group without euid=0 (i.e., as root).

6. Make 3000userlogin use the shell that is specified in the user's password entry (which is actually read from /etc/passwd). Check by setting someuser's shell to a new shell and then see if that new shell runs when you run 3000userlogin. You can change a user's shell with the chsh command.
chsh replaces the shell for the user in /etc/passwd with the one specified.
As 3000userlogin has hardcoded /bin/bash, you need to use pw_entry->pw_shell in the code in place of "/bin/bash". Then it will always invoke what's set by chsh.
7. Can you set 3000shell to be a user's default shell? What changes do you have to make for chsh to accept 3000shell? Does anything obvious break when running 3000shell this way, and how can you change 3000userlogin to fix it?
man chsh
The specified shell needs to be listed in /etc/shells. Note: if you use "sudo chsh -s" to change the shell this restriction won't apply. You need to type chsh alone as the user you're changing the shell for (this is expected, as opposed to having another superuser doing it). You'll see "UNKNOWN" instead of the username at the prompt. Just pass the environment variable "USER" as well with setenv().
8. Does a user's default shell have to be a regular shell? Could it instead be an arbitrary program? How do you know?
Just give it a try.
As mentioned, the shell is just a regular program that takes user commands for execution. In a broader sense, a GUI program (that runs other programs if the user clicks on a button or types in commands) can also be considered a shell.
9. Are the environment variables set by 3000userlogin the only environment variables that are set (visible) after you successfully login? How do you know?
3000userlogin is invoking a shell (e.g., bash). You can find out how that shell sets more. Environment variables can be set at any point in any program. Oftentimes, both the shell and the login program set some.
10. Note that 3000userlogin uses environ not envp (as an argument to main()) to access environment variables. Why not use envp? (Try changing the code to use envp and see what happens: worse or better?)
Try to think from the perspective of portability. environ is defined in unistd.h (specified in the POSIX standard), while whether the invoking program passes the envp argument is more uncertain. However, in this specific case, in terms of syntax highlighting, envp looks better as it passes \$TERM so you'll see colors with it.

Optional/advanced tasks

1. Prompt for the user's password before logging the user in (Step 2 above). To do this, you'll need to get the right password hash and then figure out how to hash the entered password to check if it matches. Recall the format of /etc/shadow and you may need to learn how to use the openssl passwd command.
If the algorithm index is 6 (in our case):
openssl passwd -6 -salt <salt_after_the_2nd\$> <user_typed_password>
to be matched with the value after the 3rd \$. To implement it, you need some programming.

Note: you are not required to complete this task and here is the code for your reference. Alternatively, you can also use the crypt() function (see: man 3 crypt).